Dense Convolutional Network (DenseNet)

Dense Convolutional Network (DenseNet)

- DenseNet, short for Densely Connected Convolutional Networks, was introduced by Gao Huang, Zhuang Liu, Laurens van der Maaten, and Kilian Q. Weinberger in 2017.

- DenseNet builds on the ideas of residual connections (ResNet) but connects each layer to every other layer in a feed-forward fashion.

- DenseNet set a new standard by being highly parameter-efficient, easier to train, and more accurate on benchmark datasets such as CIFAR and ImageNet.

Motivation: DenseNet

- In deep networks, earlier layers tend to lose information as the depth increases due to the vanishing gradient problem and redundant feature maps. ResNet mitigated this with identity-based skip connections.
- DenseNet pushes this idea further: instead of adding outputs via skip connections, it concatenates them, preserving all information and encouraging feature reuse.
- **Motivations:**
  - Improve information and gradient flow.
  - Reuse features instead of relearning them.
  - Reduce the number of parameters and computation.

Core Idea: Dense Connectivity

- In a DenseNet, each layer receives inputs from all preceding layers and passes its own feature maps to all subsequent layers.
- If there are L layers, there will be L(L+1)/2 connections, leading to dense connectivity.

- Mathematical Formulation:

  Let $x_0, x_1, \ldots, x_{l-1}$ be the outputs of previous layers.

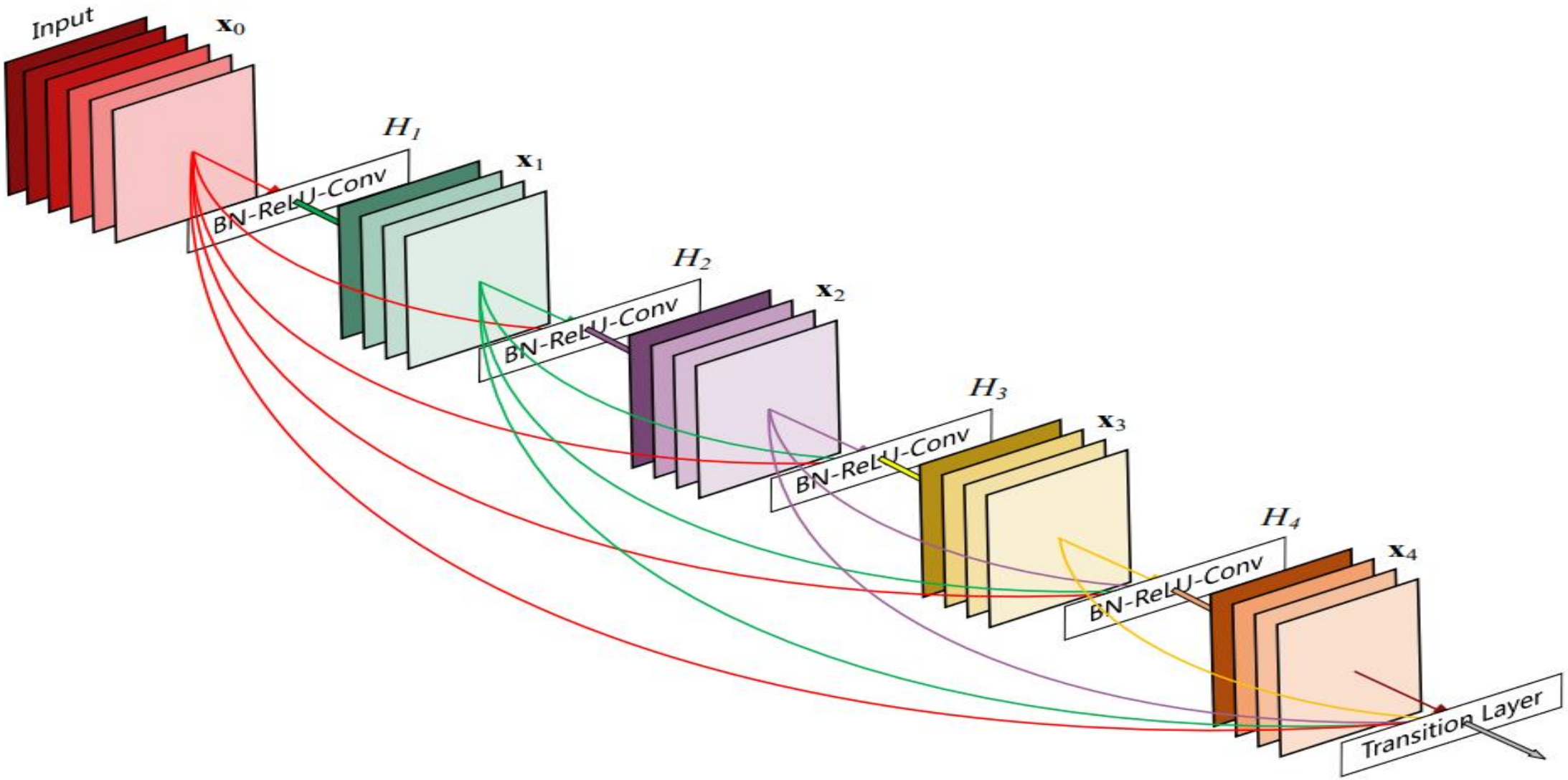
  The input to layer l is:

$$x_l = H_l ([x_0, x_1, \ldots, x_{l-1}]),$$

  where H_l is a non-linear transformation and [.] denotes concatenation.

  This promotes feature preservation and gradient flow.

Core Idea: Dense Connectivity



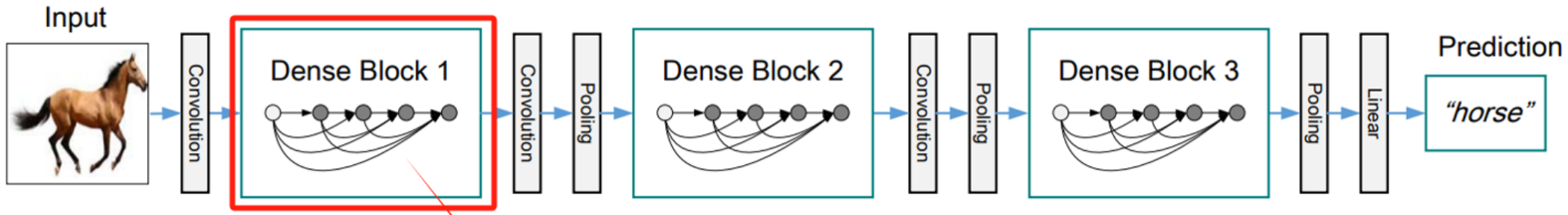
DenseNet Architecture

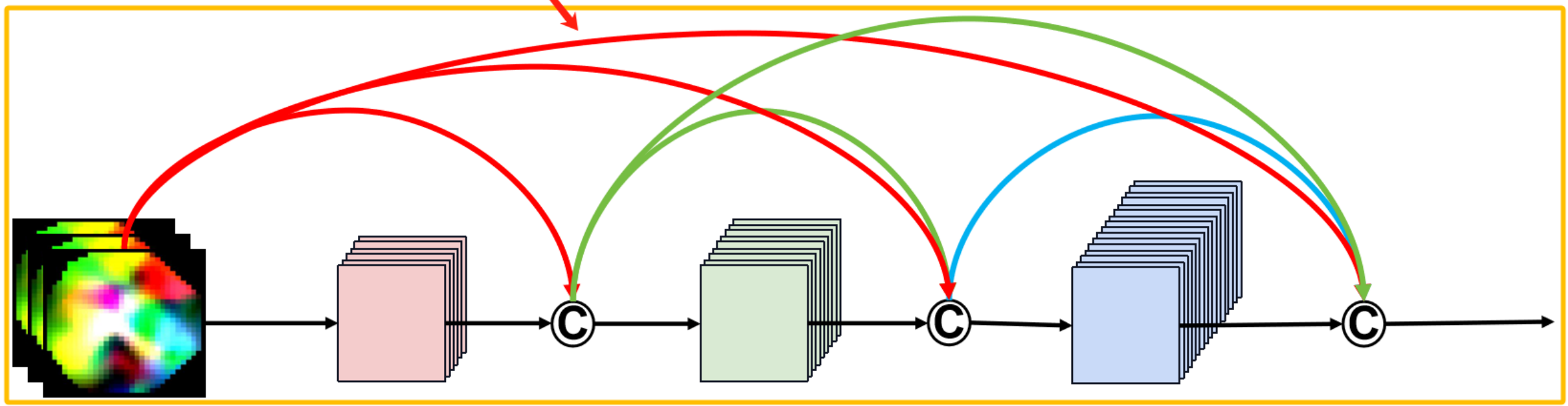
A typical DenseNet architecture consists of:

- An initial convolution layer
- Multiple Dense Blocks
- Transition Layers in between
- Final classification layer

DenseNet Architecture: Dense block

- Each block consists of multiple DenseLayers where,
  - Each layer receives inputs from all previous layers, i.,e.; each layer receives all previous feature maps
  - Feature maps are concatenated along the channel dimension.
  - Each layer's output is passed to all subsequent layers.


Input
Convolution
Dense Block 1
Convolution
Pooling
Dense Block 2
Convolution
Pooling
Dense Block 3
Pooling
Linear
Prediction
"horse"


C
C
C

DenseNet Architecture: DenseLayer

- In Basic DenseNet, each DenseLayer follows

  BN → ReLU → Conv (3×3)

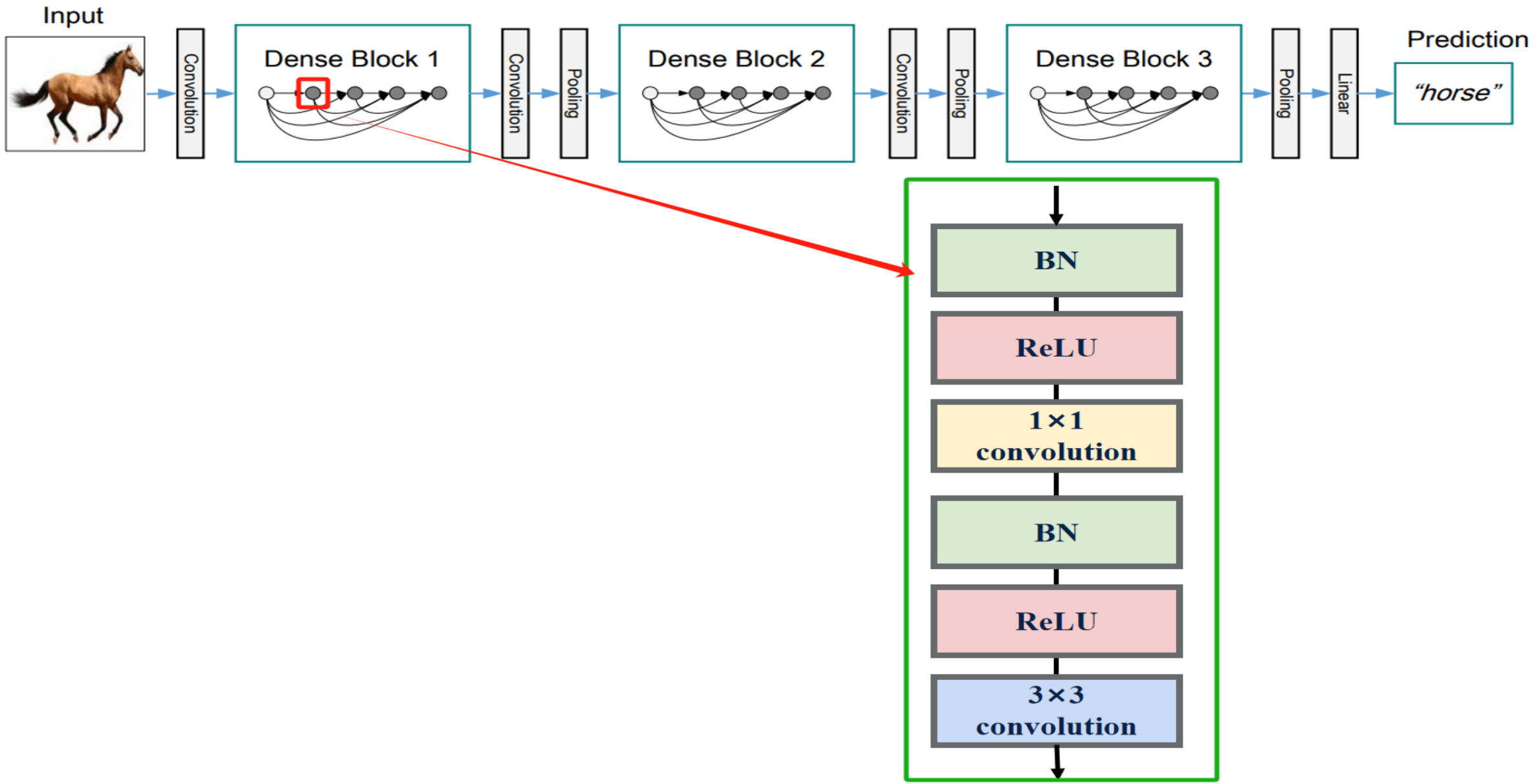
  BN – Batch Normalization

- In Bottleneck DenseNet, each DenseLayer (most commonly used):

  BN → ReLU → Conv (1×1) → BN → ReLU → Conv (3×3)

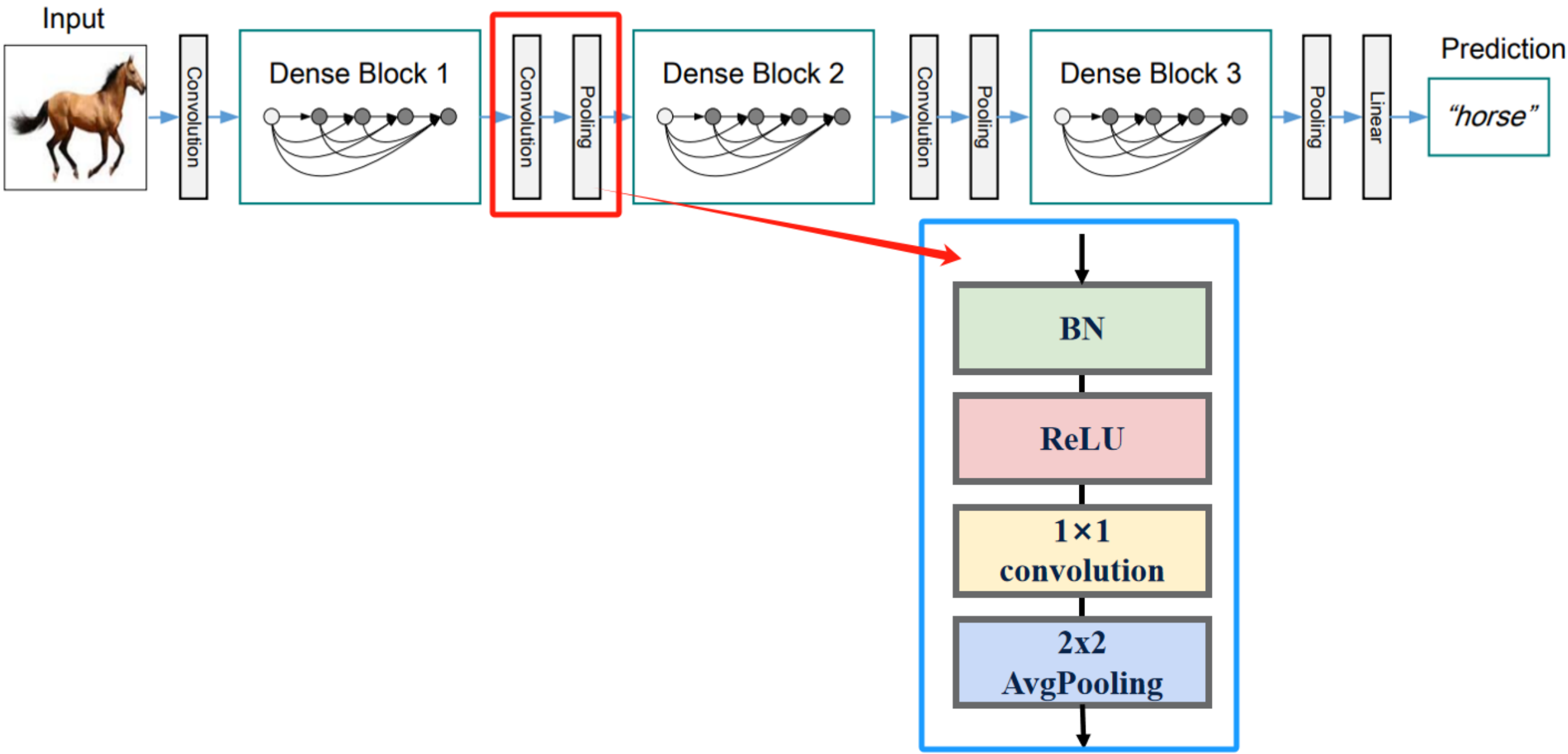
Note:

- DenseLayer usually has more inputs due to the dense connections.
- Therefore, in order to reduce the parameters, DenseLayer introduces 1×1 convolution as the bottleneck layer before the 3×3 convolution to reduce the number of input feature maps and improve the computational efficiency.


Input
Convolution
Dense Block 1
Convolution
Pooling
Dense Block 2
Convolution
Pooling
Dense Block 3
Pooling
Linear
Prediction
"horse"
BN
ReLU
1×1 convolution
BN
ReLU
3×3 convolution

DenseNet Architecture: Transition Layers

- To improve model compactness, the number of feature-maps are reduced through using 1×1 convolution and spatial resolution is reduced through downsampling at transition layer
- Used between Dense Blocks to control model complexity and resolution
- Components:
  - BatchNorm + ReLU
  - 1x1 Convolution to reduce depth
  - 2x2 Average Pooling to reduce spatial resolution


Input
Convolution
Dense Block 1
Convolution
Pooling
Dense Block 2
Convolution
Pooling
Dense Block 3
Pooling
Linear
Prediction
"horse"
BN
ReLU
1×1
convolution
2x2
AvgPooling

DenseNet Architecture: Growth Rate

- The Growth Rate (k) is a hyperparameter that determines how many new feature maps (information) each layer adds to the "global state" of the network.
- It controls the amount of new information added at each stage
- If each function H_ℓ produces k feature maps, the number of input feature maps for the ℓ-th layer is:

$$k_0 + k \times (\ell - 1), \text{ where}$$

  - k_0 : Number of feature maps/channel in the initial input layer
  - k : Growth rate (number of feature maps added per layer)
  - ℓ : Layer index

DenseNet: Advantages

- Dense Connections
- Feature Reuse
- Implicit Deep Supervision
- Parameter Efficiency
- Faster Convergence

Performance and Benchmarks

Model	Params	CIFAR-10 Acc	ImageNet Top-1
ResNet-152	60M	~93.6%	77.0%
DenseNet-121	8M	~94.2%	75.0%
DenseNet-201	20M	~95.1%	77.4%

DenseNet: Applications

- Medical imaging (e.g., chest X-ray classification)
- Scene parsing, object detection, segmentation
- Used in unsupervised/self-supervised pretraining scenarios

DenseNet: Limitations

Limitation	Description
❌ Memory Intensive	Due to concatenation, requires more GPU memory
❌ Feature Map Growth	Output channels grow linearly with depth
❌ Complexity	Harder to visualize/debug due to dense connections
❌ Slower Inference	Because of many concatenations

References:

https://abhijitmore.github.io/posts/DenseNet/#4-densenet-architecture

【Image Classification】【Deep Learning】【Pytorch Version】Detailed Explanation of DenseNet Model Algorithms-CSDN Blog